PRIVATE INTERVIEW PREPARATION

# CensusChat

## Engineering Interview Manual

How to explain the system, defend the tradeoffs,
and map the build to the Snowflake Applied AI rubric.

---

01 LEAD WITH
Deterministic SQL trust boundary

02 TEACH
Closed topology, open vocabulary

03 BE CANDID
Grounding evidence is not a runtime guarantee

SECTION 01

# The 90-second opening

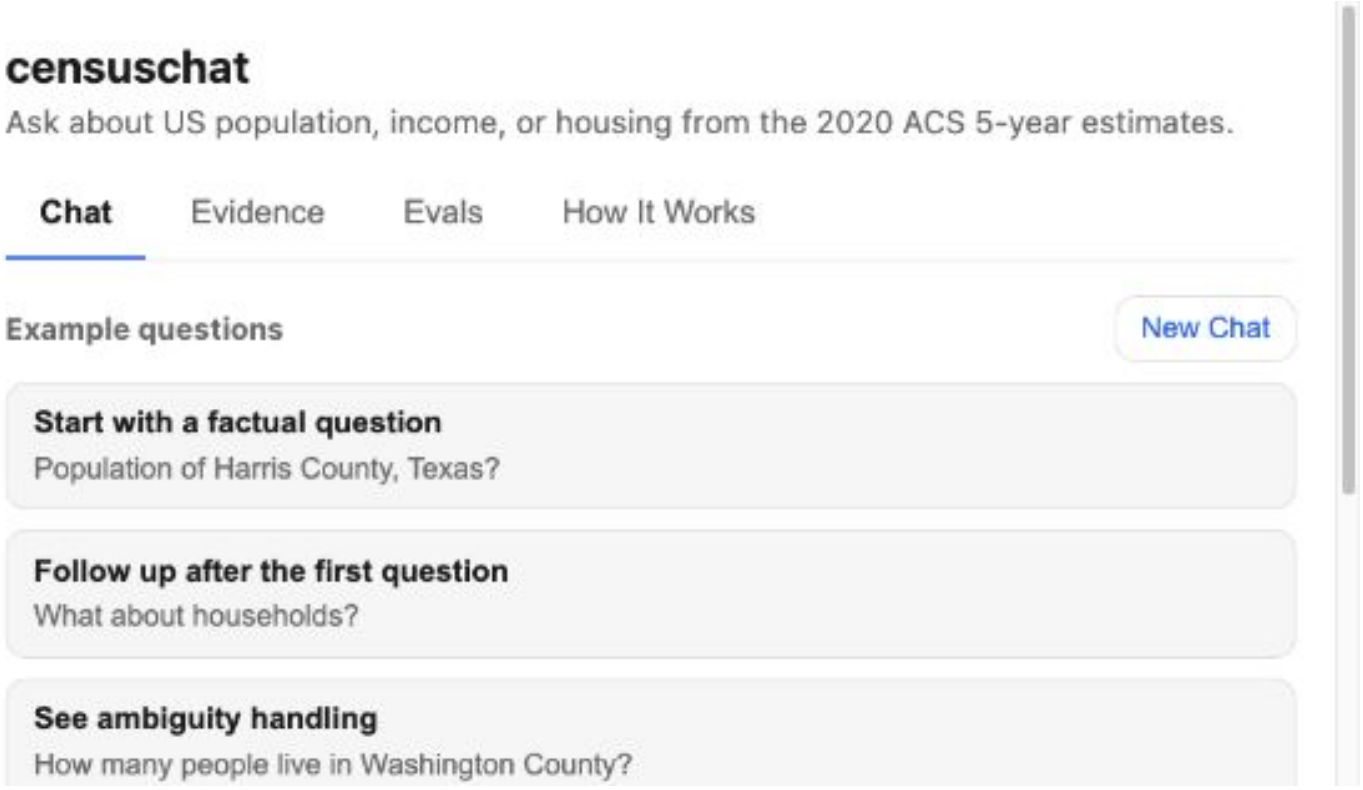
Start with the problem, the governing architecture, and one honest boundary.

**SAY THIS**

CensusChat is a production-minded chat agent for the 2020 ACS five-year estimates. It combines runtime discovery over thousands of Census variables with a small, explicit tool loop. The model can decide what to look up, but it cannot decide what SQL is safe: every request-time Snowflake query crosses a default-deny AST gate. The design optimizes for grounded answers, honest ambiguity, and observable failure under a 24-hour constraint.

## The story in three moves

- **Coverage without prompt bloat.** The schema topology is small and stable; the variable vocabulary is large and discovered at runtime through local FTS.
- **Autonomy inside deterministic boundaries.** Sonnet selects among exactly three tools, while code enforces SQL safety, unresolved geography ambiguity, retry limits, and terminal stream behavior.
- **Evidence with named limits.** Offline tests prove deterministic layers. Live evals exercise the real model and Snowflake stack. Semantic prose quality remains human-reviewed.




*Current reviewer surface: Chat, Evidence, Evals, How It Works, plus a New Chat boundary for session isolation.*

**DO NOT OVERCLAIM**

The strongest hard guarantee is SQL safety. Numeric answer grounding is a model instruction with selected offline evidence checks, not a serving-time validator for every final-answer number.

SECTION 02

# Assignment rubric map

Tie each rubric dimension to implementation evidence, then volunteer the relevant limitation.

| Dimension | Evidence | Known limit | Interview move |
|---|---|---|---|
| **LLM / AI Engineering** | Runtime variable and geography discovery; three-tool Sonnet loop; schema rules in context; deterministic ambiguity backstop; AST SQL gate. | Final-answer numbers are not all runtime-validated. Prose quality has no calibrated model judge. | Lead with the split between model reasoning and code enforcement. |
| **Production Quality** | SSE streaming; graceful degraded mode; sanitized errors; bounded recovery; SQL statement timeout; durable Evidence traces; health endpoint. | Single-instance SQLite, no rate limit or spend cap, cached Snowflake health, soft watchdog. | Explain failure behavior before listing infrastructure. |
| **Judgment Under Constraints** | 2020-only allowlist; FTS instead of embeddings; no agent framework; three tools only; decennial and Langfuse cut deliberately. | The interface arrived late in the original build. Some broader capability remains flaky or informational. | Defend what was excluded as strongly as what was built. |
| **Reflection and Self-Awareness** | The reflection names false-green tests, prompt defects, eval-calibration failures, production gaps, and a prioritized next-day plan. | Do not turn reflection into self-criticism. Use incidents to show changed engineering judgment. | Tell one concrete failure and the general lesson it produced. |

**WHY IT MATTERS** The assignment explicitly evaluates the ability to explain and defend open-ended technical decisions. A rubric answer should therefore have the form: choice, evidence, tradeoff, rejected alternative.

## A compact evidence ladder

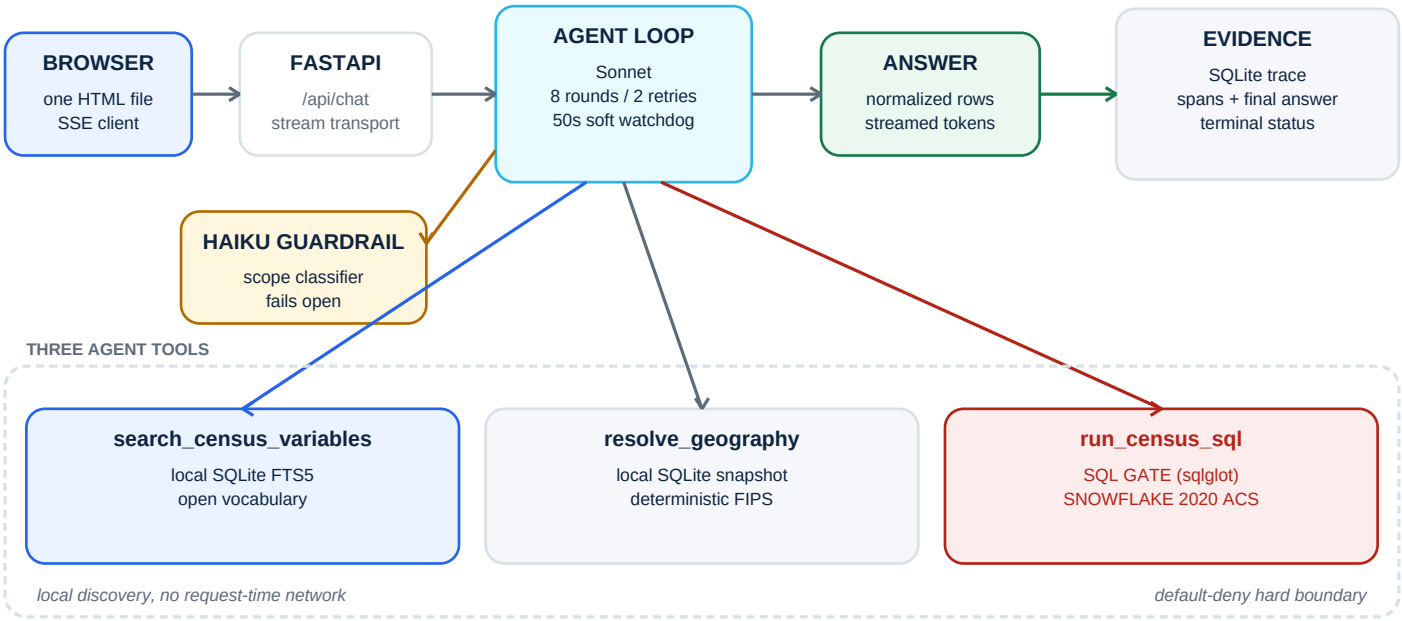
- **Code:** the strongest evidence for deterministic guarantees.
- **Offline tests:** repeatable proof that code-level contracts hold.
- **Live evals:** evidence about the integrated model, prompt, tools, and database.
- **Reflection:** evidence of judgment, not evidence that a behavior works.

`SOURCE PATHS docs/assignment.pdf pp. 2-5 | README.md | docs/reflection.md`

SECTION 03

# Architecture at a glance

The model is an orchestrator inside a narrow action space, not the security boundary.




## The governing design insight

CensusChat separates **closed topology** from **open vocabulary**. Join rules, Census block-group key structure, rollup recipes, and statistical constraints are small enough to teach in the system prompt. Thousands of variable labels and geography records stay outside the prompt and are retrieved only when needed.

> **SAY THIS** The prompt teaches how the database works, not everything the database contains. That keeps context small while preserving broad variable coverage.

## Why three tools is a feature

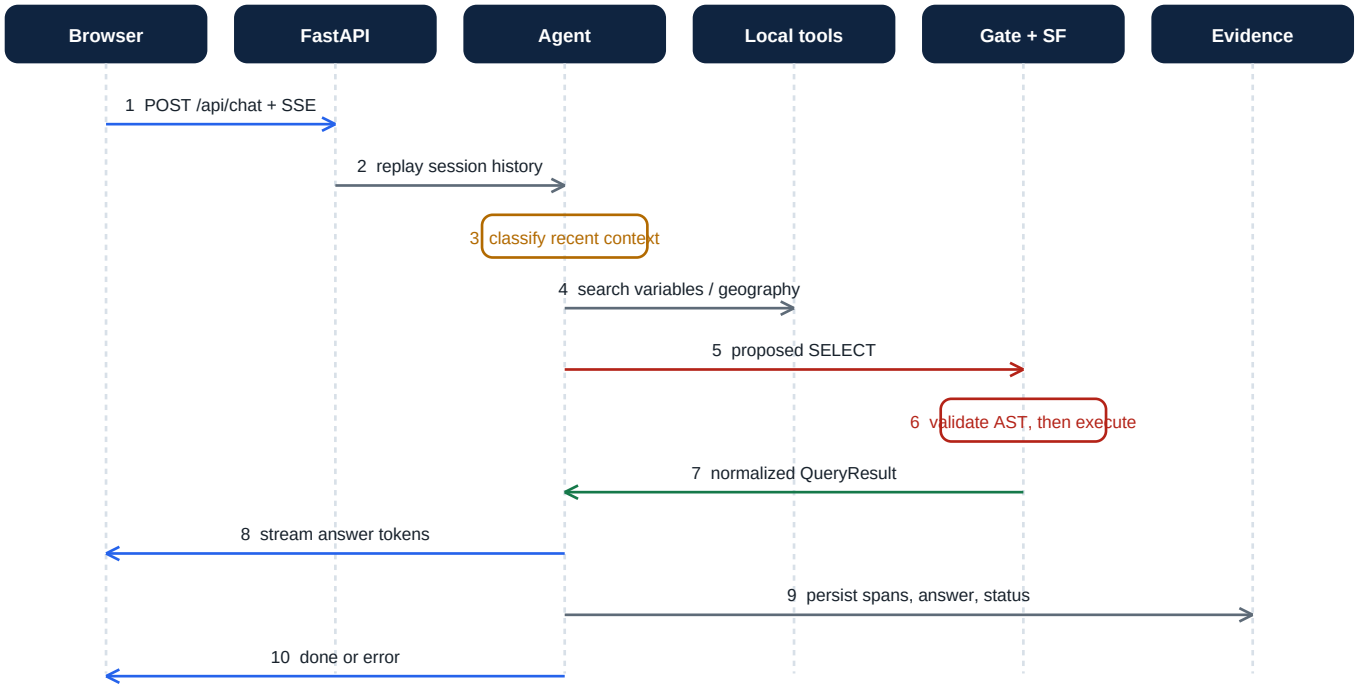
- **search_census_variables** translates user language into Census field identifiers using local SQLite FTS5.
- **resolve_geography** maps a state or county name to deterministic FIPS candidates using local SQLite.
- **run_census_sql** is the only request-time Snowflake code path, and every call crosses the SQL gate.

`SOURCE PATHS src/agent.py | src/tools.py | src/sqlgate.py | src/model_config.py`

SECTION 04

# One request, end to end

Walk through the turn in time order. This is the clearest way to explain state, tools, safety, and observability together.




## What happens at each boundary

- The browser sends a client-generated **session_id** and message to FastAPI, which returns an SSE stream.
- The agent replays the full user/assistant history from SQLite, then gives the classifier only the two most recent stored messages for context-aware routing.
- Sonnet discovers variables and geography locally, proposes SQL, receives normalized query rows, and streams answer tokens.
- Each tool call emits start/end events. Every turn terminates with **done** or **error**. A separate trace record stores spans, final answer, terminal status, and timing.

**WHY IT MATTERS**

The data flow makes the trust boundaries inspectable: user text becomes model context, never SQL string interpolation; proposed SQL becomes executable only after structural validation.

**DO NOT OVERCLAIM**

Full-history replay is simple and effective for a demo, but it can replay orphaned user turns after infrastructure failures. New Chat gives the user a fresh session boundary; a production fix would make turn persistence transactional or exclude failed turns from replay.

`SOURCE PATHS src/app.py | src/agent.py | src/sessions.py | src/tracing.py`

SECTION 05

# Data model and statistical correctness

The database stores small geographic pieces. The app can combine some measures into county or state answers, but not all of them.

## Think about the data in three steps

| | |
|---|---|
| STORE | **1. Start with the smallest unit.** Each source row describes one Census block group in the 2020 ACS five-year estimates. A block group has its own population count, household count, income statistics, and other measures. |
| ADD | **2. Add counts to answer larger-geography questions.** Population is additive. **Harris County population = sum of its block-group population counts.** The same idea works for other counts when every row measures the same universe. |
| STOP | **3. Do not combine medians.** Two block groups report median household incomes of $55k and $95k. Their average is $75k, but that does not make the county median $75k. The block groups may contain different numbers and distributions of households, so the true county median cannot be reconstructed from those two medians. |

## QUICK RULES

| Rule | Plain-English meaning |
|---|---|
| **Match the universe** | People, households, workers, and occupied housing units are different populations. |
| **Missing is not zero** | NULL becomes 'not reported.' A verified income top-code displays as $250,000 or more. |
| **Supported rollups** | The shipped source supports state and county answers, not city, ZIP, or metro boundaries. |
| **One vintage** | Only the 2020 ACS five-year release is queryable, and the SQL table allowlist enforces it. |

## Why not compare 2015-2019 with 2016-2020?

Those five-year estimates share four of the same years, and the 2020 release uses changed block-group boundaries. A result might look like a trend while mostly comparing overlapping data. Supporting time comparisons would require vintage-aware retrieval and code that rejects invalid cross-vintage analysis.

| | |
|---|---|
| SAY THIS | The key question is not just whether SQL can aggregate rows. It is whether the statistic itself is valid to combine. Counts usually are; medians are not. |

`SOURCE PATHS docs/schema-notes.md | docs/decisions.md D-003/D-005/D-008 | src/tools.py`

SECTION 06

# Trust boundaries and graceful failure

Separate advisory model behavior from code-enforced invariants. This is the strongest system-design section.

| Layer | Kind | Role | Failure behavior |
|---|---|---|---|
| **System prompt** | Soft | Quoting, aggregation, grounding, and answer-style instructions. | The model may ignore or misapply it. |
| **Haiku classifier** | Soft | Fast-fail off-topic, adversarial, and inappropriate input. | Fails open on timeout or error. |
| **Ambiguity backstop** | Hard | Blocks SQL when a geography resolved as ambiguous during the turn. | Produces deterministic clarification. |
| **SQL gate** | Hard | Parses one SELECT, checks every table, rejects banned structures, injects LIMIT. | Defaults to deny; rejected SQL never executes. |
| **Snowflake session** | Hard | 25-second statement timeout and sanitized SQL only. | Bounds database execution, not connection or model time. |

## What the SQL gate checks

- sqlglot parses with the Snowflake dialect; regex is not the parser.
- Exactly one statement, SELECT only, with CTEs allowed and DDL, DML, INTO, calls, session variables, and unverifiable table positions rejected.
- Every referenced table must match the fully qualified allowlist. Explicit columns are required; COUNT(*) is the narrow exemption.
- A LIMIT is injected when missing. User text is never interpolated into SQL.

**SAY THIS**

The classifier is allowed to fail open because it is not load-bearing. Availability stays high, while the SQL boundary still fails closed. At scale I would add defense in depth with a least-privilege Snowflake role and resource monitor.

## Bounds that prevent hanging or uncontrolled repair

- Two retries after SQL failure or zero rows; then deterministic honest failure.
- Eight total tool-loop rounds regardless of success or failure.
- A 50-second watchdog stops new rounds after the budget, but cannot cancel a call already in flight.

`SOURCE PATHS src/sqlgate.py | src/guardrail.py | src/agent.py | src/contracts.py`

SECTION 07

# State, streaming, and Evidence

Explain these as three separate responsibilities, even though the UI brings them together.

| Responsibility | Mechanism | Why |
|---|---|---|
| **Conversation state** | SQLite messages keyed by session_id | Full user/assistant replay gives multi-turn continuity and survives reloads. |
| **Transport state** | SSE ChatEvents | Tokens, tool start/end, terminal done/error, and progress reach the browser incrementally. |
| **Evidence state** | Separate SQLite trace store | Guardrail, model, and tool spans plus final answer, terminal status, timing, and raw JSON. |
| **Browser boundary** | localStorage session_id + New Chat | New Chat starts a fresh replay context without deleting old Evidence history. |

## What to point out in the Evidence tab

- The guardrail verdict and latency show whether the turn took the fast-fail path.
- Model spans expose stop reason, token counts, and time spent per round.
- Tool spans show arguments, bounded result summaries, success/failure, and latency.
- The terminal step pairs the final answer with done/error status and total timing.

**WHY IT MATTERS**

Observability is part of the reviewer story because it makes a stochastic system inspectable. It also revealed the recent session-replay bug: traces showed an older Washington County turn being replayed into a Harris County request.

**DO NOT OVERCLAIM**

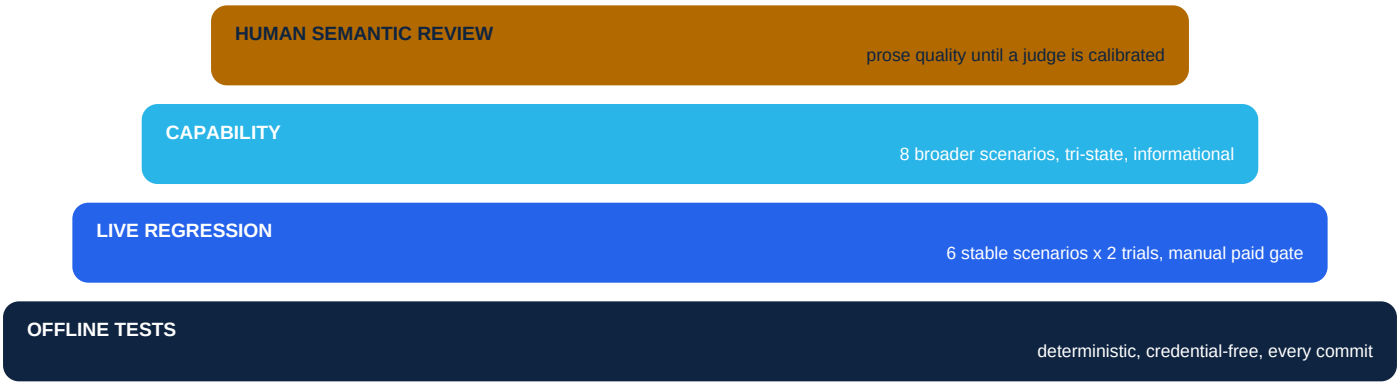
Evidence is application-local tracing, not Langfuse. It has no cross-service search, alerting, retention policy, or replica-aware aggregation. Recording is deliberately fail-soft so a tracing bug cannot break chat.

`SOURCE PATHS src/sessions.py | src/tracing.py | static/index.html | docs/decisions.md D-021/D-023/D-025`

SECTION 08

# Testing and evaluation strategy

Use the right instrument for each layer. Do not ask mocked unit tests to certify model behavior.




| Suite | Scenarios | Contract |
|---|---|---|
| **Regression (6)** | DF-05, MT-01, AMB-01, UN-01, OT-01, INJ-02 | Stable, objectively scored behaviors. Two live trials must both pass when the manual workflow is run. |
| **Capability (8)** | DF-01, CMP-01, AMB-02, PM-02, PM-03, AMB-03, UN-08, PM-08 | Broader or stochastic coverage. Pass, fail, or inconclusive, but non-blocking. |

## What deterministic graders can prove

- Expected variable and geography appear in captured tool evidence.
- Answer is nonblank or contains a stable reference value.
- Refusal paths avoid tools and use refusal language; ambiguity paths ask and avoid SQL.
- Protected median variables are not aggregated with SUM or AVG.
- Four-digit answer figures are checked only against visible final-turn query-row cells; hidden rows become inconclusive rather than passing.

**DO NOT OVERCLAIM**

The committed 14/14 artifact is from commit d44c1cc on 2026-08-06 and predates the current suite and tri-state contracts. The UI labels it legacy. It demonstrates that a real-stack run existed, not that the current branch is 14/14.

**SAY THIS**

I did not add an LLM judge because an uncalibrated judge would create a score, not evidence. Prose quality stays human-reviewed until a judge is compared with labeled examples and agreement is measured on held-out data.

`SOURCE PATHS tests/ | evals/scenarios.py | evals/run_evals.py | .github/workflows/`

SECTION 09

# Deployment and production quality

The app is deployable and observable as a controlled demo. Production at customer scale would require a different state and cost-control layer.

| Layer | Choice | Operational meaning |
|---|---|---|
| **Edge** | Native Caddy on EC2 | TLS, basic auth, SSE flushing; app port bound to loopback on the host. |
| **Application** | Dockerized FastAPI/Uvicorn | One stateless process around stateful local files; image includes frontend and committed eval artifacts. |
| **Model** | Anthropic SDK | Sonnet agent and Haiku classifier, pinned in one module. |
| **Data** | Snowflake Marketplace share | Only run_census_sql touches it at request time; local snapshot handles discovery. |
| **Persistence** | Mounted SQLite files | Sessions and traces survive restart and deploy, but do not support horizontal replicas. |
| **CI** | GitHub Actions | Credential-free offline tests on PR/main; protected, manually triggered paid regression workflow. |

## Verified deployment snapshot

**EVIDENCE** https://censuschat.brianmar.com and the local instance both returned status=ok, snapshot=ok, snowflake=ok on 2026-08-13. This is a point-in-time check, not continuous availability evidence.

## What is still missing for production

- Rate limiting, per-session spend caps, abuse monitoring, and a least-privilege Snowflake role with a resource monitor.
- Shared session/trace storage for multiple replicas, plus retention and deletion policies.
- Per-call cancellation deadlines. The current watchdog checks only between rounds.
- Live health or circuit breaking that can detect Snowflake failure after boot without violating the one-query-path invariant.
- Central telemetry with search, alerting, and deployment correlation.

**SAY THIS** I would ship this as a time-bounded reviewer demo. I would not call it multi-tenant production infrastructure until cost controls, shared state, cancellation, and centralized observability are added.

`SOURCE PATHS Dockerfile | docker-compose.yml | deploy.sh | src/health.py | .github/workflows/`

SECTION 10

# Five-minute reviewer walkthrough

Use one coherent session. Show behavior first, then open one evidence surface to explain why it happened.

| Step | Prompt/action | What it demonstrates | What to inspect |
|---|---|---|---|
| 1 | Population of Harris County, Texas? | Variable search, exact county resolution, safe state/county rollup, grounded fact. | Point to tool order and final QueryResult evidence. |
| 2 | What about households? | Full-history replay carries Harris County into the follow-up without restatement. | Show the second variable resolution and same geography. |
| 3 | How many people live in Washington County? | Thirty matches should trigger clarification, not a silent choice. | No Snowflake SQL while ambiguity is unresolved. |
| 4 | How many people will live in Texas in 2050? | The ACS is historical measurement, not projection. Fast refusal should avoid SQL. | Guardrail or answer path, zero database work. |
| 5 | Open Evidence, then Evals | Evidence explains one real turn. Evals explains test intent and result provenance. | Call out legacy benchmark labeling before the reviewer does. |

## Suggested narration

- Before the first prompt: 'I will show a fact, a context-dependent follow-up, ambiguity, and an unsupported request.'
- Before Evidence: 'The chat is the product surface; Evidence is the explanation surface.'
- Before Evals: 'The app does not pretend every behavior has the same measurement quality, so stable regression and broader capability are separate.'

**DO NOT OVERCLAIM** A live model can vary. If a scenario behaves unexpectedly, use Evidence to localize the failure and discuss the appropriate fix. Do not rerun until it happens to look good.

## Recovery line if the demo fails

**SAY THIS** This is exactly why the trace and eval layers exist. Let me show whether the failure was routing, discovery, SQL validation, execution, or answer generation, and then I will explain which layer should own the fix.

SOURCE PATHS README.md: Evaluating the running demo | static/index.html | evals/README.md

SECTION 11

# Judgment under the 24-hour constraint

The strongest answer is not 'I built everything.' It is 'I knew which guarantees were worth encoding and which breadth to cut.'

| Decision | Why it was rational | Alternative rejected |
|---|---|---|
| **FTS5 over embeddings** | Variable retrieval needed exact, local, inspectable matching. A probe showed a morphology/ranking problem, not missing semantic representation. | Embedding dependency, index build, retrieval opacity, and tuning time. |
| **Handwritten tool loop** | Exactly three tools and explicit SSE/tracing needs kept control flow small and legible. | Framework abstraction and repair behavior that would be harder to inspect. |
| **2020 only** | Older ACS release overlaps four years and uses changed CBG boundaries. Code-enforced scope prevents invalid comparisons. | Broader but misleading vintage coverage. |
| **SQL gate before feature breadth** | A single unsafe or structurally unverifiable query is a customer risk. Default-deny validation is deterministic and testable. | Prompt-only SQL safety or a regex denylist. |
| **SQLite sessions and traces** | Fast durable evidence for one demo instance with no external service dependency. | Distributed state infrastructure that the assignment did not require. |
| **No LLM judge** | Semantic scores are not trustworthy until calibrated against human labels. | A polished but unvalidated quality number. |

## Two incidents that changed the engineering approach

- **Prompt defect hidden by recovery.** Unquoted mixed-case Census identifiers failed live even while mocked tests passed. Successful self-repair initially made the systematic defect look like resilience. Lesson: recovery attempts are failure signals and real-stack evals test the product, not just the code.
- **Uncalibrated grounding check produced false reds.** Variable IDs and missing division logic were misread as fabrication. Lesson: an eval check is a measuring instrument and needs known-good calibration plus mutation tests.

**SAY THIS**

My best judgment was encoding high-cost mistakes as deterministic boundaries: unsafe SQL, unresolved geography, invalid median aggregation, bounded repair, and honest terminal behavior. I cut breadth where it would weaken those guarantees.

SOURCE PATHS `docs/reflection.md` | `docs/decisions.md` | `docs/01-architecture.md`

SECTION 12

# Likely interview questions, part I

Keep each answer under a minute. Name the tradeoff and the rejected alternative.

### Why does the classifier fail open?

Because it is an availability and UX layer, not the trust boundary. If Haiku is unavailable, a legitimate Census request should still proceed. SQL safety remains fail-closed below it, and off-topic content still cannot escape the three-tool action space. The tradeoff is that an outage may allow irrelevant turns to reach Sonnet and cost more.

### Why FTS5 instead of embeddings?

The retrieval target is a structured vocabulary of Census labels and IDs, and the observed failures were token morphology and ranking. FTS5 is local, deterministic, cheap, and inspectable. I would add embeddings only after an error analysis showed semantic recall failures that lexical retrieval could not solve.

### Why did you avoid LangChain or LangGraph?

The loop has three tools, one model vendor, eight bounded rounds, and custom streaming and trace events. A handwritten Anthropic loop makes every transition and failure rule visible. A framework would become attractive when orchestration complexity, provider portability, or durable workflow resumption exceeded this small control surface.

### Why only the 2020 ACS five-year vintage?

The older release overlaps four of five years and uses different block-group boundaries. Adding it would create plausible but invalid trend comparisons. The current allowlist makes cross-vintage SQL impossible. If multi-vintage access became necessary, I would add vintage-aware retrieval and a gate rule rejecting mixed-vintage statements.

### Why can counts aggregate but medians cannot?

Counts are additive across non-overlapping block groups. A median is an order statistic and cannot be reconstructed from subgroup medians alone. Where aggregate numerator and denominator variables exist, the system may compute a true mean and explicitly state that substitution.

### Why exactly three tools?

Each tool owns one distinct external effect: discover variables, resolve geography, execute data SQL. That is enough for the assignment while keeping the action space auditable. Adding a tool should require a new responsibility that cannot fit an existing boundary, not merely convenience.

### Why is the 50-second watchdog soft?

It is checked between agent rounds, so it can stop new tool work after the budget is observed, but it cannot interrupt an Anthropic or Snowflake call already in flight. That leaves headroom under the assignment's 60-second target without creating a guarantee. A production version would add per-call cancellation and a request-wide deadline.

**INTERVIEW PATTERN** Answer architecture questions as: requirement, choice, why it fits this system, tradeoff, threshold for revisiting the choice.

SECTION 13

# Likely interview questions, part II

These answers distinguish demonstrated behavior from unfinished production work.

### How do you prove that answers are grounded?

Today I can prove selected parts: expected tools ran, expected IDs were resolved, and visible answer figures match captured final-turn query-row cells in the eval harness. I cannot claim every final-answer number is runtime-validated. The strongest next step is to retain complete per-turn query evidence, buffer the final answer, and validate structured numeric claims before release.

### Why have an inconclusive eval outcome?

It prevents missing evidence from becoming a false pass. Bounded trace summaries may hide later query rows, so some grounding checks cannot honestly decide. Inconclusive stays non-passing and in the denominator. It is informational in capability reporting and blocking in regression trials.

### Why fourteen scenarios, and why not more?

The current bottleneck is grader quality, not case count. Six stable regression scenarios protect objectively measurable behavior; eight capability scenarios expose broader or stochastic behavior without creating merge roulette. I would expand only with cases tied to observed failures and checks that have been proven to fail correctly.

### What breaks first under real traffic?

Cost exposure and state topology. There is no rate limit or spend cap, and SQLite sessions and traces are local to one instance. Next come cancellation, stale boot-time health, and observability. I would add limits and shared state before adding replicas because replication without shared state would make behavior less coherent.

### How did you use AI coding tools?

AI produced much of the implementation and independent review agents found real defects. My responsibility was dataset recon, architecture, constraints, acceptance criteria, live verification, and deciding which suggestions to reject. The key lesson was that code review agents still missed a prompt-plus-database integration defect, so I added real-stack eval evidence rather than assuming more review would solve it.

### What would you build next?

First, serving-time numeric provenance. Second, transactional session turns so failed requests cannot remain as orphaned instructions. Third, per-call cancellation. Fourth, rate limits, spend caps, shared state, and centralized telemetry. Fifth, a genuinely independent decennial source for conflicting-answer cases.

**DO NOT OVERCLAIM**

Do not say 'production ready' without a qualifier. Say 'production-minded reviewer demo' and immediately name the scale, cost, and observability work required for customer traffic.

SECTION 14

# Interview cheat sheet

Read this page immediately before the interview. It is the compressed version of the entire manual.

## FIVE CLAIMS TO LEAD WITH

- The AST SQL gate is the hard trust boundary.
- Closed topology plus open runtime vocabulary avoids schema prompt bloat.
- Exactly three tools keep the action space narrow and auditable.
- Ambiguity, retries, and terminal behavior have code backstops.
- Tests, live evals, and human review measure different layers.

## KEY NUMBERS

- 2020 ACS five-year estimates, covering 2016-2020.
- 3 tools: variable search, geography resolution, SQL execution.
- 2 recovery retries, 8 total rounds, 50-second soft watchdog.
- 25-second Snowflake statement timeout, 200-row injected limit.
- 6 regression + 8 capability scenarios.
- 455 offline tests at the documented snapshot.

## FIVE CLAIMS NOT TO MAKE

- 'Every answer number is runtime-grounded.'
- 'The 14/14 artifact proves the current branch.'
- 'The watchdog guarantees the 60-second requirement.'
- 'Evidence is production observability.'
- 'The app is horizontally scalable or cost bounded.'

## NEXT FIVE IMPROVEMENTS

- Validate structured numeric claims before final answer release.
- Make turn persistence transactional; exclude failed orphan turns.
- Add per-call deadlines and cancellation.
- Add rate limits, spend caps, shared state, and central telemetry.
- Add an independent decennial source for conflict handling.

## File map for live discussion

| Question | Open |
|---|---|
| **How does the loop work?** | src/agent.py |
| **Where is the hard boundary?** | src/sqlgate.py |
| **How are variables and geographies found?** | src/tools.py + src/snapshot.py |
| **How is context stored?** | src/sessions.py |
| **How is a turn explained?** | src/tracing.py + Evidence tab |
| **What is actually evaluated?** | evals/scenarios.py + evals/README.md |
| **What changed and why?** | docs/decisions.md + docs/reflection.md |

**LAST LINE** The design principle is simple: let the model reason where uncertainty is useful, and move high-cost mistakes into deterministic, observable boundaries.